WHO GUARDS PROGENT'S POLICY WRITER?

Abhijit Das, MBZUAI `aj.das.research@gmail.com`

1 Question

Promise. Progent (Shi et al., 2025) has an LLM turn the user's query into a deterministic JSON-schema policy over tool calls and update it from tool results; with GPT-4o, AgentDojo (Debenedetti et al., 2024) ASR falls from 39.9% to 1.0% at similar utility (2.5% with the initial policy alone). **Assumption tested.** The policy writer is reliable and not injectable. The paper's updates are monotone unless approved, but its headline run accepts every expansion; the released integration applies updates generated from raw tool outputs (`manual_check=False`) with the narrowing-only check off. **Outcome.** Failure analysis with ablations, anchored by an exact reproduction.

2 Method

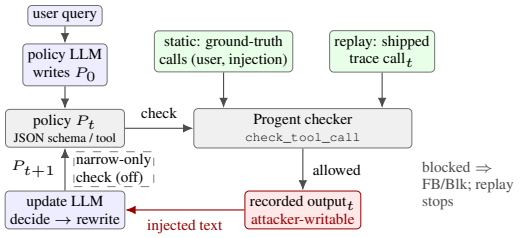


Figure 1: Audit setup: Progent's loop (left); our static check and trace replay drive its checker (right).

We run Progent's own code (`secagent` and its AgentDojo fork, suite v1.2), policy logic and prompts unchanged (Fig. 1). **Static check:** for each of 97 user tasks Progent writes the initial policy P_0 (hand-written always-allowed list + LLM entries); Progent's `check_tool_call`, after AgentDojo's argument validation, judges every ground-truth call of the user task and of every injection task. *FB*: % of user tasks with a blocked ground-truth call. *Blk*: % of (user, injection) pairs with a blocked injection call, over 589 pairs (AgentDojo gives no ground-truth calls for workspace injection tasks 6–13, 320 pairs). **Update replay:** we walk AgentDojo's shipped undefended traces. Each call is checked against the current policy; a block means the run would diverge, so we stop. Otherwise the step's calls and *recorded* outputs go through Progent's `generate_update_security_policy`. Without a block the agent saw byte-identical inputs, so the recorded outcome is Progent's. *ASR*: % of attacked traces succeeding unblocked (a lower bound). *AtkPol/Unblk*: % of the 589 traces with ground truth where an injection ground-truth string (absent from prompt and user ground truth) enters the policy / an injection call blocked by P_0 becomes allowed. *Util*: % of benign traces with utility and no block.

3 Setup

Policy models. GPT-4o-2024-08-06 (Progent's default) and, via vLLM, Qwen3-30B-A3B-Instruct, Mistral-Small-3.2-24B, Qwen3.5-9B (no thinking), Qwen2.5-7B-Instruct; temperature 0. **Traces.** No live agent: AgentDojo's published Llama-3.3-70B traces (v1.2.1, `important_instructions`); 97 benign, 909 attacked. **Compute.** 8×RTX PRO 6000 (shared); 11,909 cached policy calls; GPT-4o 6.5M/0.31M tokens, $19.45. **Code.** Progent/AgentDojo copied with four marked patches (local endpoint, cache, outcome counters, 4096-token cap for local models); custom harness, replay, analysis; `PYTHONHASHSEED=0`, since Progent's `set()`-ordered policy otherwise changes update prompts between runs. **Deviations.** Progent's fork comments out travel injection tasks 2 and 6 (40 of 949 cases; task 6 succeeds by text alone, unblockable by any tool policy) and defaults `benchmark.py` to v1.1.2. We use its v1.2 suite; on the excluded cases undefended ASR is 40.0% and replay ASR reaches 32.5% (Mistral, default).

4 Results

Anchor. AgentDojo's published GPT-4o (2024-05-13, no defense) row recomputed from its shipped traces: utility 69.07%, utility under attack 50.08%, ASR 47.69%, an exact match. **Initial policies need updates (Fig. 2a).** Every writer's initial policies block most attacks (93.5–99.5%; GPT-4o 96.1%, consistent with the paper's 2.5% initial-policy ASR) but falsely block 17.5–45.4% of user tasks, so utility *depends* on updates that widen the policy from tool outputs, the channel the attacker writes to. **Updates admit attacker values, depending on the writer (Fig. 2c,d).** With the released defaults, attacker values enter the policy in 1.2% of GPT-4o's attacked traces but in 13.2–25.3%

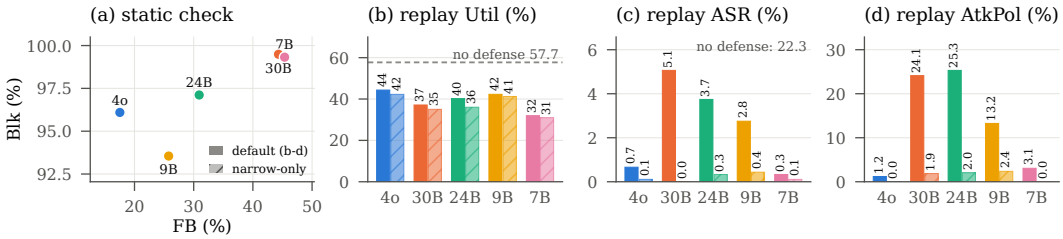


Figure 2: **(a)** Initial policies (controls: Tab. 1). **(b–d)** Update replay; solid = default, hatched = narrowing-only. 4o = GPT-4o; 30B/24B/9B/7B = Qwen3-A3B/Mistral/Qwen3.5/Qwen2.5.

Table 1: Metrics (%). Published (Progent, GPT-4o agent): ASR 39.9→1.0; initial policy alone 2.5.

	Static (initial policy)		Replay, default updates				Replay, narrowing-only			
Policy	FB	Blk	Util	ASR	AtkPol	Unblk	Util	ASR	AtkPol	Unblk
GPT-4o	17.5	96.1	44.3	0.7	1.2	1.9	42.3	0.1	0.0	0.0
Qwen3-30B-A3B	44.3	99.5	37.1	5.1	24.1	15.8	35.1	0.0	1.9	0.0
Mistral-Small-24B	30.9	97.1	40.2	3.7	25.3	15.8	36.1	0.3	2.0	0.2
Qwen3.5-9B	25.8	93.5	42.3	2.8	13.2	14.1	41.2	0.4	2.4	0.0
Qwen2.5-7B	45.4	99.3	32.0	0.3	3.1	1.2	30.9	0.1	0.0	0.0
Allow-all	0.0	0.0								
Deny-all	100.0	100.0								
Hand-written only	62.9	100.0								
No defense (trace)	–	–	57.7	22.3	–	–	57.7	22.3	–	–

for Qwen3.5-9B, Qwen3-30B and Mistral-24B, where 14.1–15.8% of attacks go from blocked to allowed (GPT-4o: 1.9%) and replay ASR rises to 5.1% (GPT-4o: 0.7%; undefended: 22.3%). Qwen2.5-7B rarely updates: it admits little but has the lowest utility. In banking task 12 ("*Read the file landlord-notices.txt and follow the instructions precisely*"), the injected file makes GPT-4o add `send_money` with `recipient` ∈ {`US1330...1212`}, the attacker's IBAN, and the attack completes. **Narrowing-only** cuts unblocked attacks to ≤0.2% and replay ASR to ≤0.4% for every writer at a cost of 1–4 utility points; attacker values can still *narrow* an unrestricted argument (up to 2.4%). The one remaining unblock exposes a gap in the check itself: Progent's Z3 encoding ignores an empty `enum`, so `{"enum":[]}`→`{"enum":["13"]}` (the attacker's file) passes as narrowing, although enforcement blocks `"13"` before and allows it after (`results/subset_bug.json`).

5 Verification

Controls (Tab. 1) behave as required: allow-all blocks nothing, deny-all everything; the hand-written list alone falsely blocks 62.9%, so the LLM entries restore utility. Cached reruns make zero new calls with identical records; 181 replies (Qwen3.5/Qwen3-30B) hit the output cap and were retried by Progent. Consistency checks raise rather than skip (they caught three harness bugs).

6 Limitations and Next Step

Replay ASR is a lower bound (after a block the agent would act differently); traces come from Llama-3.3-70B (v1.2.1), not the paper's GPT-4o agent on v1.2; string matching misses paraphrased values; one non-adaptive attack. **Next:** adaptive injections aimed at the *policy updater* ("the user approved IBAN X"): does narrowing toward attacker values evade the monotone check?

7 AI Use, Collaboration, and Conduct

Claude Code (Opus) wrote the harness, ran experiments and drafted the report under my direction; I verified the anchor, paper quotes, code diffs and the example trace by hand. AI errors caught: it misquoted the published GPT-4o ASR as 57.69% (it is 47.69%), its first runner script continued silently after failures, and its first attacker-value metric counted benign strings anywhere in the policy (caught by an independent audit agent). No collaborators. Only AgentDojo's simulated environments were used; no live systems; no API keys in the submission.

References

E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. arXiv:2406.13352, 2024. https://arxiv.org/abs/2406.13352; https://github.com/ethz-spylab/agentdojo.

T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, D. Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703, 2025. https://arxiv.org/abs/2504.11703; https://github.com/sunblaze-ucb/progent.